

PROYECTO API II

Sistema Gestión de Tickets para el Reporte y
Seguimiento de Fallas en equipos de Laboratorio de
Computo

Sergio Olivas, Miriam Ruiz, Camila
Martínez, Otman Guiza, Alicia
Santiago, Mariana Sandoval.

1. Introducción

Este proyecto está dirigido al personal técnico y administrativo encargado del mantenimiento de los laboratorios de cómputo de la universidad, así como a los docentes de estos laboratorios donde se hace uso de dichos equipos durante sus clases.

1.1 Planteamiento del problema

El reporte de fallas en los equipos de los laboratorios de cómputo se realiza de manera informal y no centralizada, principalmente por correo electrónico o de manera verbal. Esto ocasiona que no haya un seguimiento adecuado de estas incidencias, generando retrasos en sus reparaciones.

Además, no se cuenta con un registro histórico que permita identificar que equipos presentan fallas recurrentes, que técnico atendió cada caso y el tiempo que tomo resolver cada incidencia.

1.2 Idea principal

El proyecto es una aplicación web diseñada para crear y gestionar los reportes de fallas técnicas en los equipos de laboratorios de cómputo de la universidad.

Los reportes se generarían mediante tickets de soporte a través de un formulario, con seguimiento desde su creación hasta su resolución. También, permitirá la asignación de los responsables, el seguimiento de los estados y el registro histórico de las incidencias.

1.3 Objetivo general

Desarrollar e implementar un sistema web de gestión de tickets que permita a los usuarios de los laboratorios de cómputo reportar fallas de equipo y dar seguimiento a su resolución, mientras que el personal técnico pueda gestionar, priorizar, asignar y documentar la atención de dichos reportes, funcionando dentro de la red local de la universidad con el hardware disponible.

1.4 Objetivos específicos

1. Implementar un formulario para la creación de tickets que capture el nombre del ticket, aula o laboratorio, equipo afectado (equipos de cómputo, proyector, etc.), tipo de problema y descripción.
2. Desarrollar un panel de gestión donde el personal responsable pueda visualizar, priorizar, actualizar y cerrar tickets.
3. Mantener un historial de los reportes generados, incluyendo su estado y los cambios realizados a lo largo del tiempo.
4. Garantizar que el sistema funcione correctamente dentro de la red local de la universidad y sea utilizable desde dispositivos móviles mediante un diseño responsivo.

1.5 Alcance del proyecto

El sistema contempla un CRUD de tickets relacionados con las fallas en equipos de los laboratorios de cómputo. También contempla:

- Inicio de sesión y administración de usuarios.
- Control de acceso mediante roles.
- Formulario de creación de tickets (nombre del ticket, aula, equipo afectado, tipo de problema y descripción).
- Asignación de un responsable a cada ticket.
- Seguimiento del estado del ticket.
- Notificaciones de creación y estado a ambas partes.
- Uso mediante dispositivos móviles.
- Historial de reportes.
- Respaldo periódico de la información.

1.6 Restricciones del proyecto

- **Equipo:** 6 personas.
- **Tiempo:** semestre 2026-2, estructurado en 3 etapas.
- **Requisitos técnicos obligatorios:** inicio de sesión, interacción con base de datos, administración de usuarios, mínimo 90% de efectividad, manejo de corte de acceso a la base de datos, sistema de respaldo.
- **Evaluación:** el profesor funge como cliente durante cada avance.
- **Presupuesto:** \$0, utilizando herramientas y servicios gratuitos.
- **Complejidad técnica del proyecto:** grande.
- Debe operar en la red local institucional.

1.7 Justificación

La finalidad de desarrollar esta aplicación es para agilizar y centralizar el proceso de reporte y atención de fallas de estos equipos de cómputo.

Mediante esta plataforma web, la información de cada incidencia podrá mantenerse organizada y centralizada, donde se guarden los datos del servicio, como las fechas de reporte y atención, o que personal fue asignado a este equipo.

2. Marco de referencia

2.1 Conceptos clave para el desarrollo del sistema

ADMINISTRACIÓN DE PROYECTOS INFORMÁTICOS II

- **Sistema de gestión de tickets:** es una herramienta utilizada para registrar, organizar, asignar y dar seguimiento a solicitudes o incidencias. En este proyecto, cada falla reportada generará un ticket que podrá ser gestionado hasta su resolución.
- **Ticket:** es el registro digital de una incidencia. Contendrá información como el aula o laboratorio, equipo afectado, tipo de problema, descripción, responsable, estado y seguimiento.
 - **Usuarios y roles:** son las personas que interactúan con el sistema y cuentan con roles, los cuales permitirán establecer que acciones puede realizar cada usuario dentro del sistema. En este sistema se contemplan los siguientes roles: administrador, líder / encargado del laboratorio, técnico y usuario final (Maestro).
- **Laboratorio:** es el espacio físico en el que se encuentran los equipos, cada ticket debería estar asociado a un laboratorio.
- **Arquitectura y stack:**
 - **Base de datos:** la información del sistema será almacenada mediante PostgreSQL utilizando Supabase. En ella se almacenarán los usuarios, laboratorios, tickets, estados, asignaciones e información relacionada con las incidencias.
 - **Autenticación:** será gestionada mediante Supabase Auth, encargándose de procesos como la creación de sesiones y la verificación de correo electrónico.
 - **Interfaz de Programación de Aplicaciones (API):** gestiona la lógica del negocio y comunica el frontend con los servicios de datos. En este proyecto se utilizará FastAPI para implementar el backend y gestionar la comunicación con los servicios necesarios.
 - **Notificaciones:** se utilizará el servicio SMTP de Supabase para enviar correos electrónicos relacionados con la verificación de cuentas, creación de tickets y cambios de estado.
 - **Diseño responsivo:** permite que las interfaces del sistema se adapten a diferentes tamaños de pantalla, incluyendo computadoras y dispositivos móviles.

2.2 Audiencia

Audiencia	Rol en el proyecto	Categoría de audiencia
Administrador del sistema	Gestiona los usuarios, roles y supervisa el funcionamiento general del sistema	Interno - Supporter
Líder/Encargado del laboratorio	Gestiona los tickets de su laboratorio y los asigna a técnicos de su equipo.	Interno - Driver
Técnico de soporte	Atiende y cierra los tickets asignados.	Interno - Driver
Docentes	Genera tickets y consulta el estado de sus reportes.	Interno - Observador
Universidad	Provee y administra la red local donde operará el sistema	Interno - Supporter

Equipo de desarrollo	Diseñan programan y prueban el sistema	Interno - Supporter
Profesor de la materia	Funge como cliente durante la evaluación	Externo – Driver

2.2 Requerimientos funcionales

- El sistema debe permitir el inicio de sesión mediante sus credenciales.
- El sistema debe de diferenciar entre administrador, líder, técnico y usuario final.
- El sistema debe permitir la creación de tickets a través de un formulario que capture el nombre del ticket, aula, equipo afectado, tipo de problema y descripción.
- El sistema debe permitir asignar un responsable a cada ticket generado.
- El sistema debe contar con un panel de gestión donde el personal técnico pueda visualizar, priorizar, actualizar y cerrar tickets.
- El sistema debe permitir el seguimiento del estado del ticket a lo largo de su ciclo de vida (Abierto, en proceso, cerrado).
- El sistema debe enviar notificaciones al usuario y al técnico sobre la creación y los cambios de estado de un ticket.
- El sistema debe mantener un historial de reportes generados, consultable por equipo o por aula.
- El sistema debe permitir al administrador gestionar altas, bajas y modificaciones de usuarios y roles.
- El sistema debe generar respaldos periódicos de la información almacenada en la base de datos.

2.2 Requerimientos no funcionales

- El sistema debe operar dentro de la red local institucional sin depender de acceso a internet externo.
- El sistema debe controlar el acceso a la información mediante roles y permisos.
- El sistema debe manejar adecuadamente los cortes de acceso a la base de datos, evitando pérdida de información.
- La interfaz debe adaptarse a dispositivos móviles y computadoras.
- El sistema debe garantizar la confidencialidad de la información de los usuarios mediante el control de acceso por roles.

3. Desarrollo del proyecto

3.1 Estimación del proyecto

El proyecto se dividirá en tres etapas principales ya definidas:

- **Etap 1:** Inicio del semestre – 9 de septiembre de 2026 — Definición de requerimientos y diseño (diagramas y vistas).

- **Etapa 2:** 9 de septiembre – 21 de octubre de 2026 (aprox. 5 semanas) — Desarrollo del sistema (login, base de datos, formulario de tickets, panel de gestión).
- **Etapa 3:** 21 de octubre – 24 de noviembre de 2026 (aprox. 6 semanas) — Pruebas, respaldo de información, ajustes finales y entrega.

2.2 Work Breakdown Structure (WBS)

1.0 Sistema Web de Gestión de Tickets para Laboratorios de Cómputo

• 1.1 Etapa 1: Planificación, requerimientos y diseño

- 1.1.1 Presentación de idea y sorteo de fechas
- 1.1.2 Definición de requerimientos funcionales y no funcionales
- 1.1.3 Boceto del calendario del proyecto
- 1.1.4 Diseño de diagramas (flujo, red)
- 1.1.5 Diseño de la Base de Datos (Identificación de las entidades, Modelo relacional, Esquema)
- 1.1.6 Diseño Visual (Selección de paleta de colores, creación de iconos, definición de tipografías)
- 1.1.7 Diseño de Vistas/UI (Pantalla de inicio de sesión, Dashboard, Gestión de tickets, Reportes)

• 1.2 Etapa 2: Desarrollo

• 1.2.1 Base de Datos (Supabase - PostgreSQL)

- 1.2.1.1 Creación del proyecto en Supabase y obtención de credenciales de conexión
- 1.2.1.2 Creación de esquema de base de datos inicial (Creación de tablas, relaciones y restricciones)

• 1.2.2 Backend (Python, FastAPI, SQLAlchemy)

- 1.2.2.1 Configuración del entorno virtual y conexión a la base de datos (SQLAlchemy)
- 1.2.2.2 Definición de Modelos y Esquemas (Pydantic/SQLAlchemy para Roles, Usuarios, Tickets, etc.)
- 1.2.2.3 Integración de Supabase Auth para Autenticación y Autorización (Manejo de sesiones, verificación de correos, roles)
- 1.2.2.4 Desarrollo de la API REST para CRUD de tickets, usuarios, aulas y equipos
- 1.2.2.5 Lógica de negocio (enrutamiento automático de tickets, cambio de estados)
- 1.2.2.6 Configuración de notificaciones automáticas por correo electrónico vía Supabase SMTP

• 1.2.3 Frontend (Vite: HTML, CSS, JS)

- 1.2.3.1 Estructura base del proyecto con Vite y configuración inicial
- 1.2.3.2 Desarrollo HTML/CSS de vista: Inicio de sesión
- 1.2.3.3 Desarrollo HTML/CSS de vista: Dashboard
- 1.2.3.4 Desarrollo HTML/CSS de vista: Panel de Superusuario (Altas/Bajas)

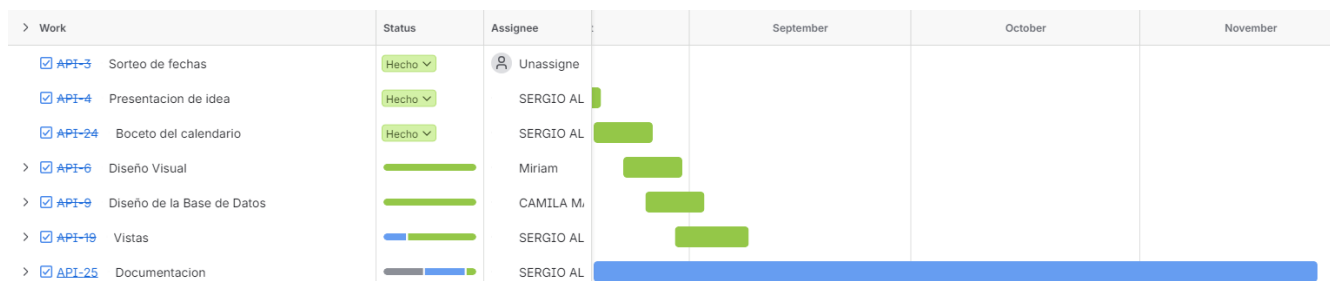
ADMINISTRACIÓN DE PROYECTOS INFORMÁTICOS II

- 1.2.3.5 Desarrollo HTML/CSS de vista: Gestión de tickets (Formulario, asignación, actualización)
- 1.2.3.6 Desarrollo HTML/CSS de vista: Reportes e Historial
- 1.2.3.7 Implementación de diseño responsivo (adaptable a móviles)
- **1.2.4 Integración (Frontend <-> Backend)**
 - 1.2.4.1 Integración de la API REST (FastAPI) con el cliente Vite (Manejo de peticiones HTTP/Fetch)
 - 1.2.4.2 Almacenamiento y manejo de tokens de autenticación en el cliente
- **1.3 Etapa 3: Pruebas, despliegue y cierre**
 - 1.3.1 Pruebas de testing (integración de la API y pruebas de UI)
 - 1.3.2 Configuración del sistema de respaldos de la base de datos (Supabase backups)
 - 1.3.3 Ajustes finales y corrección de errores (bugs)
 - 1.3.4 Despliegue del Frontend y Backend en entorno de producción
 - **1.3.5 Documentación y Entrega Final**
 - 1.3.5.1 Introducción y Marco de referencia
 - 1.3.5.2 Documentación del Diagrama de redes
 - 1.3.5.3 Documentación de Vistas o interfaces
 - 1.3.5.4 Tutorial del sistema
 - 1.3.5.5 Guía de instalación
 - 1.3.5.6 Documentación de Calendario
 - 1.3.5.7 Conclusiones

3.2 Calendario

El calendario fue gestionado con la plataforma jira:

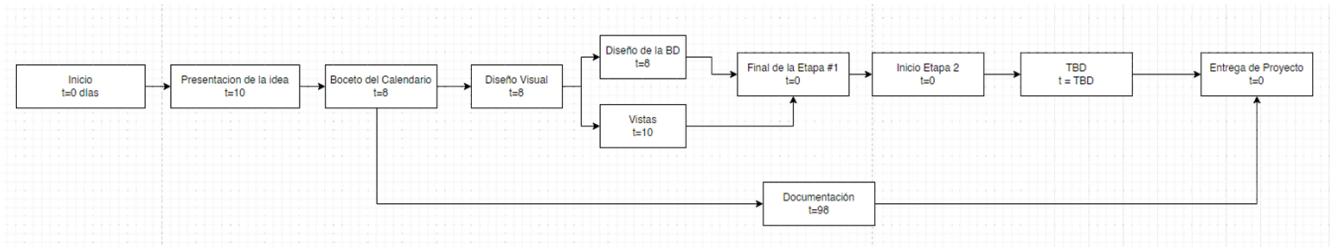
<https://proyectopds2.atlassian.net/jira/core/projects/API/timeline?rangeMode=WEEKS&atlOrigin=eyJpIjojZDRmM2UzN2FiYTFlNDlmOWI5Mjk1ODRhMDM1ZjEyOGEiLCJwIjoiajI9>



3.3 Diagramas

Como parte del desarrollo del proyecto se contemplan los siguientes diagramas:

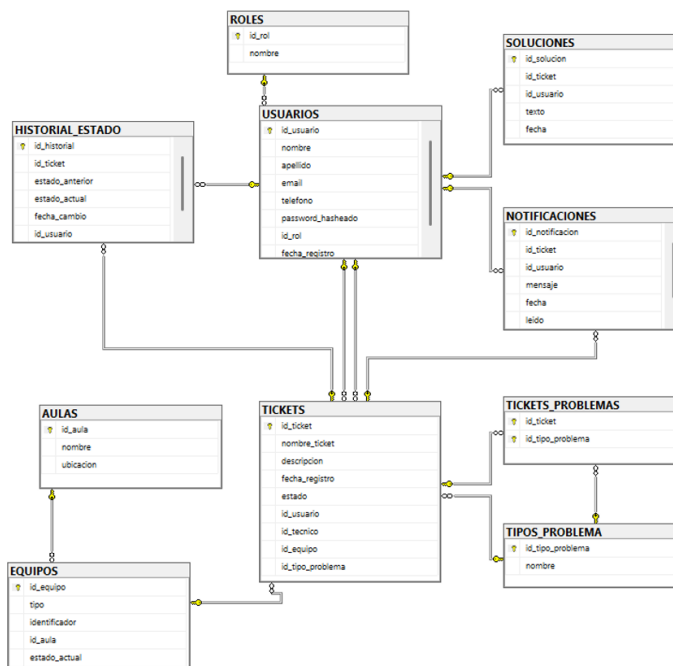
- Diagrama de red:



- Diagrama de flujo:

<https://www.figma.com/design/hlWJsJQJKxQ549AmTcVCdQ/Ticket?m=auto&t=EzB95KbKHDJrjSuY-6>

- Diagrama de la base de datos



3.3 Vistas

El sistema cuenta con las siguientes vistas, de acuerdo con las funciones de cada usuario:

<https://www.figma.com/design/hlWJsJQJKxQ549AmTcVCdQ/Ticket?m=auto&t=EzB95KbKHDJrjSuY-6>

4. Guía de instalación

Incluir requerimientos de instalación del sistema

5. Tutorial del funcionamiento del sistema

Incluir requerimientos de instalación del sistema

6. Pruebas del testing

Incluir requerimientos de instalación del sistema

7. Conclusión

Incluir requerimientos de instalación del sistema

8. Referencias

Incluir requerimientos de instalación del sistema